Guía · 2026

Code review sin pelearse

Revisa en orden, comenta para que se entienda y recibe críticas sin tomarlas personal.

Basado en Google Engineering Practices y Conventional Comments



Chamba Lab · 2026



Otro par de ojos antes de que tu código entre

Alguien lee tu cambio y decide si deja el código mejor que antes.

- Se hace sobre un pull request (PR), antes de mezclarlo a `main`
- No busca código perfecto: busca código **mejor**
- Revisas el código, nunca a la persona



Si los tests ya pasan, ¿para qué otra persona lee tu PR?

Pista: los tests solo prueban lo que alguien pensó probar.

Para qué sirve: lo que un test no ve



Dos que leen aprenden los dos

- **El diseño:** si el cambio encaja en el sistema o lo complica
- **Lo que nadie probó:** el cliente que vuelve atrás, el carrito vacío
- **Lo que se entiende:** nombres claros y comentarios que dicen por qué
- **El conocimiento:** ya no hay una sola persona que entiende el carrito
- **Lo que se enseña:** el revisor puede mostrar algo nuevo, y el autor también

Anatomía de un PR: el #42 de Rakim, por partes

Rakim es dev junior en **BodegaYa**, una app de pedidos para bodegas de Lima. Su PR agrega cupones de descuento al carrito.

`Título` › `Descripción` › `Diff` › `Tests` › `Revisores`

qué hace — qué y por qué — lo que cambia — la prueba — quién lee

Título

Una orden corta: «Aplica el cupón al total del carrito». «Fix bug» no dice nada.

Descripción

El problema, la solución y por qué así. Lo que el revisor no sabe.

Diff

3 archivos, 87 líneas: `carrito.js`, su test y un texto.

Tests

Prueban el cupón válido, el vencido y el que se aplica dos veces.

Revisores

El zorro, que conoce el carrito. La hámster de QA lo prueba después.

Tres veredictos: cómo cierras una revisión en GitHub

Comment

Dejas comentarios sin aprobar ni bloquear. Para preguntar o opinar a medio camino.

`question` `thought`

A MEDIO CAMINO

Approve

El cambio está listo para mezclarse. Puede llevar comentarios menores que el autor resuelve después.

`LGTM` `nit`

EL MÁS COMÚN

Request changes

Hay algo que el autor debe resolver antes de mezclar: un bug, un riesgo, un diseño que no va.

`issue` `blocking`

SOLO SI HACE FALTA

LGTM = *Looks Good To Me*, «a mí me parece bien». Vale solo si de verdad lo leíste.

Quién participa: cuatro miradas sobre el PR #42



La hámster de QA encuentra lo que nadie vio

Autor · Rakim
Escribe el cambio, lo describe y **arregla el PR**. Arreglarlo es su trabajo, no del revisor.

Revisor · el zorro
Lee cada línea que le asignan y responde en **un día hábil como máximo**. Si no entiende algo, pregunta.

CI · los robots
Tests, linter y formateador corren solos. Que el robot discuta las comas, **no las personas**.

QA · la hámster
Prueba el comportamiento como lo usaría el cliente: el cupón vencido, el doble clic.

Cómo se revisa: en tres pasadas, de lo general al detalle

1 · Vista general
¿debe existir?

→

2 · Lo principal
donde está la lógica

→

3 · El resto
archivo por archivo

1 · Lee la descripción

¿El cambio tiene sentido? Si la idea no va, **dilo ya y con cortesía**, antes de comentar comas.

2 · Abre carrito.js

Ahí vive el descuento. Si el diseño falla, **avisa enseguida**: puede cambiar todo lo demás.

3 · Recorre lo demás

En el orden de la herramienta. A veces conviene **leer los tests primero** para saber qué debe pasar.

Qué mirar: de lo que cuesta más arreglar a lo que cuesta menos

Diseño › **Función** › **Complejidad** › **Tests** › **Nombres** › **Estilo**

¿encaja? ¿hace lo que dice? ¿se entiende? ¿fallarían? ¿dicen qué es? al linter

Lo caro, primero

Un diseño equivocado tira abajo el PR entero. Una coma, no.

Sin futurología

No pidas código para lo que «quizás» haga falta: complejidad sin uso.

Tests que sirven

¿Fallaría el test si alguien rompe el descuento? Si no, no protege nada.

Y lo bueno

Si algo está bien hecho, dilo. También enseña.

Un comentario útil: etiqueta, asunto y por qué

PR #42 · carrito.js · línea 18

```
issue (blocking): el cupón se descuenta dos veces

Si el cliente vuelve al carrito, aplicarCupon()
corre otra vez y el total baja de nuevo.
Con BODEGA10 y S/ 24, cobramos S/ 4.

suggestion: guarda cupon.usado en el pedido
y sal de la función si ya es true.
```

Etiqueta: qué tipo de comentario es (`issue`, `suggestion`...).

Decoración: si bloquea o no, entre paréntesis.

Asunto: el problema en una línea, sobre el código.

Discusión: por qué importa y qué propones.

Las etiquetas: que se sepa qué es obligatorio y qué no

Hay que resolver

Un problema real o una tarea pendiente antes de mezclar.

`issue` `todo` `chore`

SUELEN BLOQUEAR

Propone

Una mejora, una duda o una idea. Explica qué y por qué.

`suggestion` `question` `thought`

SE CONVERSA

Pulido

Detalles de gusto o una errata. Google lo marca con «Nit:».

`nitpick` `typo` `polish`

NO BLOQUEAN

Reconoce

Lo que está bien hecho, o algo que conviene saber.

`praise` `note`

AL MENOS UN PRAISE

Formato: `etiqueta (decoración): asunto` · decoraciones: `blocking`, `non-blocking`, `if-minor`

El mismo hallazgo: dos formas de decirlo

Suena a ataque

```
¿Por qué hiciste esto así?
Obvio que está mal. Rehazlo.
```

Habla de **ti**, no del código. No dice qué falla, ni por qué, ni cómo seguir.

Rakim se defiende y nadie arregla el bug

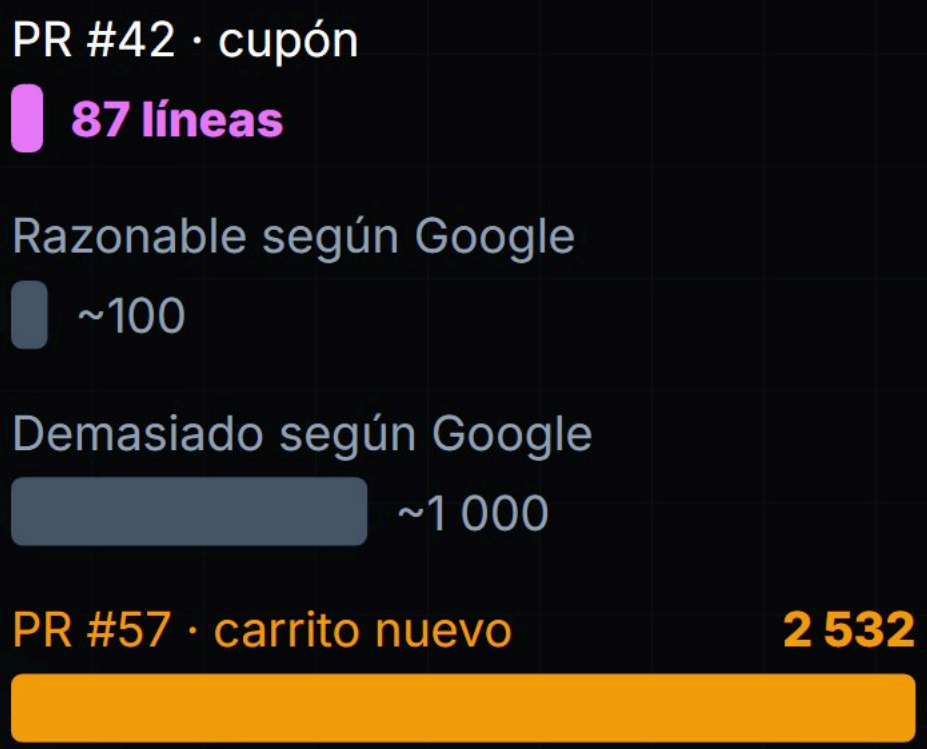
Se puede usar

```
question: ¿aplicarCupon() corre de
nuevo si el cliente vuelve al carrito?
Si es así, el total baja dos veces.
```

Habla del **código**, explica el riesgo y deja espacio para que Rakim responda.

Rakim lo prueba, lo confirma y lo arregla

PR pequeños: un cambio que se entiende solo



- Se revisa **más rápido y más a fondo**
- Mete menos bugs y es más fácil de revertir
- Si lo rechazan, pierdes poco trabajo
- Separa el refactor de la funcionalidad: dos PR, no uno

La medida no son las líneas: es **un cambio autocontenido**, que se entiende y funciona solo.

Recibir críticas: el comentario es sobre el código



Los gatos no se impresionan.
Igual algo útil dicen

No es personal
Pregúntate qué punto útil hay detrás, aunque el tono no ayude, y **responde a eso**.

Nunca en caliente
Si te molesta, sal a caminar antes de escribir. **Un mensaje en caliente se queda** en el PR.

Arregla el código, no el hilo
Si no entendieron algo, **aclara el código** o deja un comentario en él: la respuesta en el hilo no la lee el próximo.

Si no estás de acuerdo
Explica tu razón y el contexto que el otro no tiene. **Busca consenso**, no ganar.

El LGTM de 3 minutos: 2 532 líneas que nadie leyó



Nadie revisa 2 532 líneas en 3 minutos. Un LGTM así no es una revisión: es un sello.

```
PR #57  feat: carrito nuevo
46 archivos cambiados  +2143  -389

zorro aprobó estos cambios · hace 3 min
  "LGTM"

# dos días después, en producción
pedido 8812   subtotal S/ 24.00
cupón BODEGA10 aplicado 2 veces
total cobrado S/ 4.00
```

¿Apruebo o bloqueo? según lo que encontraste

	Approve + comentarios	Request changes
Un bug, un riesgo o datos en juego	No: se mezclaría roto	**Sí**, con `issue (blocking)`
Gusto o estilo	**Sí**, con `nitpick`	No: es tu preferencia
Mejora el código, pero no es perfecto	**Sí**: mejor, no perfecto	No: frenas sin motivo
Más de ~1 000 líneas	No: nadie lo leyó entero	**Sí**: pide dividirlo

Aprueba si el PR deja el código mejor y lo que queda es pulido. Bloquea solo por bugs, riesgos o un diseño caro de deshacer.

Lo que te llevas

- Revisa en orden: si debe existir, lo principal, el resto
- Comenta el código, no a la persona, y di si bloquea
- PR pequeños: así nadie aprueba sin leer

```
> guía: google.github.io/eng-practices/review
> etiquetas: conventionalcomments.org
> repo: github.com/Chamba-Lab
```